

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
INDEX

Week No	Date	Sl. No.	Type	Topic	Title with download Link for document	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records containing: - ModelID - Accuracy - Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. Calculate average model accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling	1
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	A streaming platform maintains movie records containing: - MovieID - Rating - Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. 5. Normalize rating values.	5

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Title	Data Manipulation and Analysis using the Starwars Dataset in R																					
Problem Statement	Basic Data Manipulation with Pandas: Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.																					
Objectives	- To understand how to create a Pandas DataFrame from a Python dictionary. - To learn column selection techniques using Pandas. - To apply filtering conditions on DataFrame rows. - To create and populate new columns in a DataFrame. - To sort data based on column values. - To perform basic statistical analysis using Pandas aggregate functions.																					
Dataset URL Link / File	<table><tr><td>Name</td><td>Age</td><td>City</td></tr><tr><td>Alice</td><td>23</td><td>New York</td></tr><tr><td>Bob</td><td>30</td><td>London</td></tr><tr><td>Charlie</td><td>28</td><td>Paris</td></tr><tr><td>David</td><td>35</td><td>Tokyo</td></tr><tr><td>Emma</td><td>22</td><td>Sydney</td></tr></table>				Name	Age	City	Alice	23	New York	Bob	30	London	Charlie	28	Paris	David	35	Tokyo	Emma	22	Sydney
Name	Age	City																				
Alice	23	New York																				
Bob	30	London																				
Charlie	28	Paris																				
David	35	Tokyo																				
Emma	22	Sydney																				
Functions Details	Sl. No.	Function	Package	Description of function																		
	1	DataFrame()	pandas	Creates a DataFrame																		
	2	sort_values()	pandas	Sorts DataFrame rows																		
	3	mean()	pandas	Calculates the average value																		
	4	sort_values()	pandas	Sorts DataFrame rows																		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library and create a dictionary containing Name, Age and City Data Step 2 Convert the dictionary into a pandas DataFrame Step 3 Select only the Name and Age columns from the DataFrame. Step 4 Filter the DataFrame to display individuals whose age is greater than 25.																					

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Step 5 Add a new column named Country and assign a default value of India. Step 6 Sort the DataFrame in descending order based on the Age column. Step 7 Calculate the average age of all individuals using the mean() function. Step 8 – Display all intermediate and final results.
Program	<pre># Load Necessary Packages import pandas as pd # Step 1: Create Dataset dataframe1 = { "Name": ["Alice", "Bob", "Charlie", "David", "Emma"], "Age": [23, 30, 28, 35, 22], "City": ["New York", "London", "Paris", "Tokyo", "Sydney"] } # Step 2: Create DataFrame df = pd.DataFrame(dataframe1) print("Original DataFrame") print(df) print() # Step 3: Select Name and Age Columns name_age = df[["Name", "Age"]] print("Name and Age") print(name_age) print() # Step 4: Filter Rows where Age > 25 filtered_df = df[df["Age"] > 25] print("Age > 25") print(filtered_df) print()</pre>

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

```
# Step 5: Add Country Column
df["Country"] = "India"

print("Country Added")
print(df)
print()

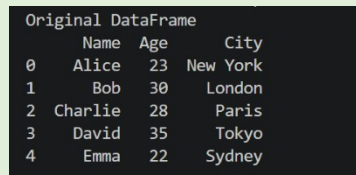
# Step 6: Sort by Age in Descending Order
sorted_df = df.sort_values(by="Age", ascending=False)

print("Sorted Data")
print(sorted_df)
print()

# Step 7: Calculate Mean Age
mean_df = df["Age"].mean()

print("Mean Age")
print(mean_df)
```

**Output
Snapshots
With
Explanation/
Observations**

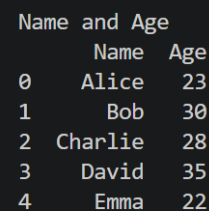


	Name	Age	City
0	Alice	23	New York
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo
4	Emma	22	Sydney

Figure 1.1 Original Dataframe

Figure 1.1 displays the original DataFrame created from a Python dictionary containing information about five individuals. The DataFrame consists of three columns: Name, Age, and City, where each row represents a different individual and their corresponding details. The `pd.DataFrame()` function converts the dictionary data into a structured tabular format, making it easier to store, view, and manipulate data using Pandas. This DataFrame serves as the starting dataset for all subsequent operations such as column selection, filtering, adding new columns, sorting, and calculating statistical measures.

Figure 1.2 shows the DataFrame after selecting only the Name and Age columns from the original dataset using Pandas column indexing. This operation extracts the required columns and creates a new DataFrame named `name_age`, removing the City column from the view. Such column selection helps focus on relevant attributes for analysis and reduces unnecessary data processing.



	Name	Age
0	Alice	23
1	Bob	30
2	Charlie	28
3	David	35
4	Emma	22

Figure 1.2 Selected Name and Age Columns

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Name	Age	City
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo

Figure 1.3: Filtered Data (Age > 25)

satisfying the condition are retained in the new DataFrame filtered_df. Figure 1.3 operation demonstrates how Pandas can be used to extract specific records based on user-defined criteria, which is a common requirement in data analysis and data preprocessing tasks.

Figure 1.4 displays the DataFrame after adding a new column named Country with a default value of "India" for all records. This operation is performed using Pandas column assignment, where the same value is assigned to every row in the newly created column. Adding new columns is a common data transformation technique used to enrich datasets with additional information.

Figure 1.4 output confirms that the Country column has been successfully added to the DataFrame, and each individual has been assigned the default value India. The enriched dataset is now ready for further operations such as sorting and statistical analysis.

	Name	Age	City	Country
0	Alice	23	New York	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
3	David	35	Tokyo	India
4	Emma	22	Sydney	India

Figure 1.4: DataFrame with Country Column

	Name	Age	City	Country
3	David	35	Tokyo	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
0	Alice	23	New York	India
4	Emma	22	Sydney	India

Figure 1.5 Sorted Data by Age (Descending Order)

the largest or smallest values. Figure 1.5 output confirms that the DataFrame has been successfully sorted in descending order of Age, placing the oldest individual at the top and the youngest individual at the bottom. Such sorting operations are commonly used in data analysis to rank, compare, and organize records efficiently.

Figure 1.6 presents the average age of all individuals in the dataset, calculated using the Pandas mean() function. The average (mean) age is obtained by summing all the age values and dividing the total by the number of individuals in the DataFrame. This operation demonstrates the use of aggregate statistical functions in Pandas for summarizing numerical data. Figure 1.6 output confirms that the Pandas mean() function has successfully computed the average age of the five individuals as 27.6 years. Such aggregate functions are widely used in data analysis to summarize and interpret numerical information efficiently.

Mean of the average age 27.6

Figure 1.6 Average Age of Individuals

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Conclusions	This experiment successfully demonstrated the use of Pandas for basic data manipulation and analysis. A DataFrame containing the details of five individuals was created using the columns Name, Age, and City. Various DataFrame operations were then performed, including selecting specific columns, filtering rows based on a condition, adding a new column with default values, sorting records, and calculating statistical measures. These operations showcased the flexibility and efficiency of Pandas in handling tabular data. The results showed that: • The Name and Age columns could be extracted independently for focused analysis. • Filtering enabled the identification of individuals whose Age was greater than 25 years. • A new Country column was successfully added to enrich the dataset. • Sorting arranged the records in descending order of age, making it easy to identify the oldest and youngest individuals. • The average age of the individuals was calculated as 27.6 years, demonstrating the use of aggregate functions in data analysis. Overall, this exercise provided a strong foundation in DataFrame creation, data selection, filtering, transformation, sorting, and statistical analysis using Pandas. These fundamental data manipulation techniques are widely used in Data Science, Machine Learning, Data Analytics, and Business Intelligence applications, making them essential skills for working with real-world datasets.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab link /Githab link) https://colab.research.google.com/drive/1sjdIVacPTNg2JO_nGS8gic99hGEtMuRe?usp=sharing 5. Class lecture link https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

Title	Data Manipulation and Analysis using the Starwars Dataset in R				
Problem Statement	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.				
Objectives	• Understand how AI model performance data is stored and represented in a Pandas DataFrame. • Filter machine learning models based on conditions, such as accuracy greater than 90%. • Learn how to select specific columns such as ModelID and Accuracy. • Sort AI models according to their accuracy values. • Calculate statistical values such as average, minimum, and maximum model accuracy. • Rank machine learning models based on their performance.				
Dataset URL Link / File	ModelID	Algorithm	PreviousAccuracy	Accuracy	
	M101	Logistic Regression	84.5	87.2	
	M102	Decision Tree	87.0	89.5	
	M103	Random Forest	90.2	93.8	
	M104	Support Vector Machine	88.5	91.4	
	M105	K-Nearest Neighbors	82.5	85.6	
	M106	Random Forest	91.0	94.5	
	M107	Decision Tree	85.5	88.3	
	M108	Neural Network	92.0	96.1	
	M109	Logistic Regression	86.0	90.8	
	M110	Neural Network	93.1	95.4	
	M111	Gradient Boosting	89.4	92.7	
	M112	Gradient Boosting	91.8	94.1	
Functions Details	Sl. No.	Function	Package	Description of function	
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the AI model data dictionary.	
	2	pd.read_csv()	Pandas	Reads AI model records from a CSV file.	
	3	dropna()	Pandas	Removes rows containing missing or invalid values	
	4	drop_duplicates()	Pandas	Removes duplicate records based on ModelID.	

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

	5	sort_values()	Pandas	Sorts the models according to their accuracy values.
	6	mean()	Pandas	Calculates the average accuracy of all AI models.
	7	groupby() and agg()	Pandas	Groups models by algorithm and calculates performance statistics.
	8	to_csv()	Pandas	Saves the analysed model results into a CSV file.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import the Pandas package. 2. Create the AI model dataset containing ModelID, Algorithm, PreviousAccuracy and Accuracy. 3. Convert the dataset into a Pandas DataFrame. 4. Clean the dataset by removing missing values, duplicate ModelIDs and invalid accuracy values. 5. Select only the ModelID and Accuracy columns. 6. Filter the models whose Accuracy is greater than 90%. 7. Add a Status column with the default value "Production Ready". 8. Change the status to "Needs Improvement" for models having Accuracy below 90%. 9. Sort the models in descending order based on Accuracy. 10. Assign ranks to the models, where the highest-accuracy model receives Rank 1. 11. Calculate the improvement of each model using: Improvement = Accuracy – PreviousAccuracy 12. Calculate the average accuracy of all models. 13. Group the models by Algorithm and calculate minimum, maximum and average accuracy. 14. Identify the best-performing, lowest-performing and most-improved models. 15. Display all the analysis results in the console. 16. Generate an automated model evaluation report. 17. Save the analysed results in a CSV file and the report in a text file.			
Program	<pre>import sys from pathlib import Path import pandas as pd from analysis_core import (analyse_models, clean_data, create_algorithm_summary, create_report, create_sample_data, perform_basic_operations)</pre>			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
#
=====
=====
# HEADING FUNCTION
# This keeps console output neat and readable.
#
=====
=====
def print_heading(title):
    print("\n" + "=" * 72)
    print(title)
    print("=" * 72)

#
=====
=====
# MAIN CONSOLE PROGRAM
#
# Run with sample data:
# python console_app.py
#
# Run with CSV data:
# python console_app.py sample_ai_models.csv
#
=====
=====
def main():
    print_heading("AI MODEL PERFORMANCE ANALYSIS -
    CONSOLE MODE")

    csv_file = None

    # If the user gives a file path, use that CSV.
    if len(sys.argv) > 1:
        csv_file = sys.argv[1]

    if csv_file:
        csv_path = Path(csv_file)

        if not csv_path.exists():
            print(f"Error: File not found: {csv_path}")
            return

        print(f"Using CSV file: {csv_path}")
        raw_data = pd.read_csv(csv_path)

    else:
        print("Using built-in sample data.")
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
raw_data = create_sample_data()

try:
    cleaned_data = clean_data(raw_data)
except ValueError as error:
    print(f"Data error: {error}")
    return

analysed_data = analyse_models(
    cleaned_data,
    production_threshold=90.0
)

results = perform_basic_operations(
    analysed_data
)

print_heading("ORIGINAL CLEANED DATA")
print(
    cleaned_data.to_string(index=False)
)

print_heading("1. SELECT MODELID AND ACCURACY")
print(
    results["selected_columns"]
    .to_string(index=False)
)

print_heading(
    "2. FILTER MODELS WITH ACCURACY GREATER THAN
90%"
)
print(
    results["models_above_90"][
        ["ModelID", "Algorithm", "Accuracy"]
    ].to_string(index=False)
)

print_heading(
    "3. ADD STATUS WITH DEFAULT "PRODUCTION READY"
)
print(
    results["status_table"]
    .to_string(index=False)
)

print_heading("4. SORT MODELS BY ACCURACY")
print(
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
results["sorted_models"][[
    ["ModelID", "Algorithm", "Accuracy"]
].to_string(index=False)
)

print_heading("5. CALCULATE AVERAGE MODEL
ACCURACY")
print(
    f'Average model accuracy: '
    f'{results["average_accuracy"]:.2f}%'
)

print_heading("MODEL RANKING")
print(
    analysed_data[
        [
            "Rank",
            "ModelID",
            "Algorithm",
            "PreviousAccuracy",
            "Accuracy",
            "Improvement",
            "Status"
        ]
    ].to_string(index=False)
)

print_heading("ALGORITHM PERFORMANCE COMPARISON")
algorithm_summary = create_algorithm_summary(
    analysed_data
)

print(
    algorithm_summary.to_string(index=False)
)

print_heading("ACCURACY IMPROVEMENT TRACKING")
print(
    analysed_data[
        [
            "ModelID",
            "PreviousAccuracy",
            "Accuracy",
            "Improvement",
            "ImprovementStatus"
        ]
    ].to_string(index=False)
)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
report = create_report(
    analysed_data,
    production_threshold=90.0
)

print_heading("AUTOMATED EVALUATION REPORT")
print(report)

# Save analysed data.
analysed_data.to_csv(
    "console_ai_model_results.csv",
    index=False
)

# Save the report.
with open(
    "console_ai_model_report.txt",
    "w",
    encoding="utf-8"
) as report_file:
    report_file.write(report)

print_heading("FILES CREATED")
print("console_ai_model_results.csv")
print("console_ai_model_report.txt")

if __name__ == "__main__":
    main()
```

**Output
Snapshots
With
Explanation/
Observations**

```
=====
ORIGINAL CLEANED DATA
=====
ModelID      Algorithm      PreviousAccuracy  Accuracy
M101  Logistic Regression      84.5      87.2
M102  Decision Tree           87.0      89.5
M103  Random Forest           90.2      93.8
M104  Support Vector Machine    88.5      91.4
M105  K-Nearest Neighbors      82.5      85.6
M106  Random Forest           91.0      94.5
M107  Decision Tree           85.5      88.3
M108  Neural Network          92.0      96.1
M109  Logistic Regression      86.0      90.8
M110  Neural Network          93.1      95.4
M111  Gradient Boosting        89.4      92.7
M112  Gradient Boosting        91.8      94.1
```

Figure 2.1 Original Cleaned Dataset

The ModelID and Accuracy columns were selected successfully. Models having accuracy greater than 90% were filtered from the dataset.

The program successfully loaded and cleaned the AI model performance dataset. The dataset contained ModelID, Algorithm, PreviousAccuracy and Accuracy.

```
=====
1. SELECT MODELID AND ACCURACY
=====
ModelID  Accuracy
M108     96.1
M110     95.4
M106     94.5
M112     94.1
M103     93.8
M111     92.7
M104     91.4
M109     90.8
M102     89.5
M107     88.3
M101     87.2
M105     85.6
```

Figure 1.2 Selected ModelID and Accuracy

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
=====
2. FILTER MODELS WITH ACCURACY GREATER THAN 90%
=====
```

```
ModelID      Algorithm  Accuracy
M108         Neural Network  96.1
M110         Neural Network  95.4
M106         Random Forest   94.5
M112         Gradient Boosting  94.1
M103         Random Forest   93.8
M111         Gradient Boosting  92.7
M104         Support Vector Machine  91.4
M109         Logistic Regression  90.8
```

Figure 1.3 Models Accuracy Greater than 90%

A Status column was added. Models with accuracy greater than or equal to 90% were marked as “Production Ready”, while the remaining models were marked as “Needs Improvement”.

```
=====
3. ADD STATUS WITH DEFAULT "PRODUCTION READY"
=====
```

```
ModelID  Accuracy  Status
M108     96.1    Production Ready
M110     95.4    Production Ready
M106     94.5    Production Ready
M112     94.1    Production Ready
M103     93.8    Production Ready
M111     92.7    Production Ready
M104     91.4    Production Ready
M109     90.8    Production Ready
M102     89.5    Needs Improvement
M107     88.3    Needs Improvement
M101     87.2    Needs Improvement
M105     85.6    Needs Improvement
```

Figure 1.4 Status Column and Models sorted by accuracy

```
=====
5. CALCULATE AVERAGE MODEL ACCURACY
=====
```

```
Average model accuracy: 91.62%
```

```
=====
MODEL RANKING
=====
```

```
Rank ModelID      Algorithm  PreviousAccuracy  Accuracy  Improvement  Status
1    M108         Neural Network  92.0        96.1        4.1    Production Ready
2    M110         Neural Network  93.1        95.4        2.3    Production Ready
3    M106         Random Forest   91.0        94.5        3.5    Production Ready
4    M112         Gradient Boosting  91.8        94.1        2.3    Production Ready
5    M103         Random Forest   90.2        93.8        3.6    Production Ready
6    M111         Gradient Boosting  89.4        92.7        3.3    Production Ready
7    M104         Support Vector Machine  88.5        91.4        2.9    Production Ready
8    M109         Logistic Regression  86.0        90.8        4.8    Production Ready
9    M102         Decision Tree   87.0        89.5        2.5    Needs Improvement
10   M107         Decision Tree   85.5        88.3        2.8    Needs Improvement
11   M101         Logistic Regression  84.5        87.2        2.7    Needs Improvement
12   M105         K-Nearest Neighbors  82.5        85.6        3.1    Needs Improvement
```

Figure 1.5 Average Model Accuracy

The average accuracy of all models was calculated. Accuracy improvement was also calculated by subtracting PreviousAccuracy from the current Accuracy.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

=====

ALGORITHM PERFORMANCE COMPARISON

=====

Algorithm	ModelCount	MinimumAccuracy	MaximumAccuracy	AverageAccuracy	AverageImprovement
Neural Network	2	95.4	96.1	95.75	3.20
Random Forest	2	93.8	94.5	94.15	3.55
Gradient Boosting	2	92.7	94.1	93.40	2.80
Support Vector Machine	1	91.4	91.4	91.40	2.90
Logistic Regression	2	87.2	90.8	89.00	3.75
Decision Tree	2	88.3	89.5	88.90	2.65
K-Nearest Neighbors	1	85.6	85.6	85.60	3.10

Figure 1.6 Models Grouped according to their algorithm and other metrics

The models were grouped according to their algorithms, and minimum, maximum and average accuracy values were calculated for each algorithm.

```
SUMMARY
Total models analysed: 12
Production threshold: 90.0%
Average model accuracy: 91.62%
Production-ready models: 8
Models needing improvement: 4

BEST MODEL
Model ID: M108
Algorithm: Neural Network
Accuracy: 96.10%

LOWEST-PERFORMING MODEL
Model ID: M105
Algorithm: K-Nearest Neighbors
Accuracy: 85.60%

MOST IMPROVED MODEL
Model ID: M109
Algorithm: Logistic Regression
Improvement: 4.80 percentage points

BEST ALGORITHM
Algorithm: Neural Network
Average accuracy: 95.75%
```

The program identified the best-performing model, lowest-performing model, most-improved model and best-performing algorithm.

Figure 1.7 Final Output

Conclusions

The exercise successfully demonstrated how Pandas can be used to analyse AI model performance data. The program selected, filtered, sorted and ranked the models, calculated average accuracy, compared algorithm performance and tracked accuracy improvement. It also generated an automated evaluation report and saved the final results in CSV and text formats.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018
2. Python Program ([https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%201/Excercise Program Console.py](https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%201/Excercise%20Program%20Console.py))
3. Class lecture link ([https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1Ut4qVMUy/view?usp=drive link](https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1Ut4qVMUy/view?usp=drive_link))

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Title	Student Marks Data Manipulation and Analysis using Pandas																																	
Problem Statement	Data Cleaning and Preprocessing with Pandas: Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling.																																	
Objectives	● To understand how to create a Pandas DataFrame containing missing values. ● To learn how to identify and handle missing data using the fillna() function. ● To remove incomplete records using the dropna() function. ● To convert a DataFrame column to the required data type using the astype() function. ● To normalize numerical data using the Min-Max Scaling technique. To understand the importance of data preprocessing before performing data analysis and machine learning tasks.																																	
Dataset URL Link / File	<table><tr><th>Name</th><th>Age</th><th>City</th></tr><tr><td>Arjun</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Priya</td><td>28</td><td>Mumbai</td></tr><tr><td>Rohan</td><td>35</td><td>Delhi</td></tr><tr><td>Sneha</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Amit</td><td>40</td><td>Bangalore</td></tr><tr><td>Deepa</td><td>30</td><td>Chennai</td></tr><tr><td>Rahul</td><td>np.nan</td><td>np.nan</td></tr><tr><td>np.nan</td><td>33</td><td>Kolkata</td></tr><tr><td>np.nan</td><td>29</td><td>Ahmedabad</td></tr></table>				Name	Age	City	Arjun	np.nan	np.nan	Priya	28	Mumbai	Rohan	35	Delhi	Sneha	np.nan	np.nan	Amit	40	Bangalore	Deepa	30	Chennai	Rahul	np.nan	np.nan	np.nan	33	Kolkata	np.nan	29	Ahmedabad
Name	Age	City																																
Arjun	np.nan	np.nan																																
Priya	28	Mumbai																																
Rohan	35	Delhi																																
Sneha	np.nan	np.nan																																
Amit	40	Bangalore																																
Deepa	30	Chennai																																
Rahul	np.nan	np.nan																																
np.nan	33	Kolkata																																
np.nan	29	Ahmedabad																																
Functions Details	Sl. No.	Function	Package	Description of function																														
	1	DataFrame()	pandas	Creates a DataFrame from structured data																														
	2	Mean()	pandas	Calculates the average value																														
	3	fillna()	pandas	Replaces missing values with a specified value.																														
	4	dropna()	pandas	Removes rows containing missing values.																														

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library. Step 2 Create a Python dictionary containing StudentID, Name, and Marks for five students. Step 3 Convert the dictionary into a Pandas DataFrame and display the original dataset. Step 4 Select and display only the Name and Marks columns. Step 5 Filter and display students whose Marks are greater than 70. Step 6 Add a Grade column by assigning grades A to E according to the marks obtained. Step 7 Display the updated DataFrame containing the Grade column. Step 8 Sort the DataFrame by Marks in ascending order and display the result. Step 9 Calculate and display the average marks using the mean() function.
Program	<pre># Import the Pandas library import pandas as pd # Create a dictionary containing student details dictionary_1 = { "StudentID": [10, 11, 12, 13, 14], "Name": ["Hasnain", "John", "Alice", "Sahil", "Cook"], "Marks": [45, 86, 90, 67, 97] } # Convert the dictionary into a Pandas DataFrame dataframe = pd.DataFrame(dictionary_1) # Display the original dataset print("Displaying the Original Dataset\ n") print(dataframe, "\n") # Select and display only the Name and Marks columns only_name_colum = pd.DataFrame(dataframe, columns=["Name", "Marks"]) print("Displaying only name and marks columns\n") print(only_name_colum, "\n") # Filter students whose marks are greater than 70 marks_greater_than_70 = pd.DataFrame(dataframe[dataframe["Marks"] > 70]) print("Displaying students with marks greater than 70\n") print(marks_greater_than_70, "\n") # Assign grades based on the marks obtained dataframe["Grade"] = ["A" if marks >= 90 else "B" if marks >= 80 else "C" if marks >= 70 else "D" if marks >= 60 else "E" for marks in dataframe["Marks"]]</pre>

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

```
]

# Display the updated dataset with the Grade column
print("Displaying the updated dataset\n")
print(dataFrame, "\n")

# Sort the dataset by Marks in ascending order
sort_by_marks = dataFrame.sort_values(
    by="Marks", ascending=True
)
print("Displaying the sorted dataset\n")
print(sort_by_marks, "\n")

# Calculate and display the average
marks average =
dataFrame["Marks"].mean() print("Average
Marks \n")
print(average)
```

Displaying the Original Dataset

```
***
  StudentID   Name  Marks
0         10  Hasnain    45
1         11   John    86
2         12   Alice    90
3         13   Sahil    67
4         14   Cook    97
```

Figure 2.1 Original Student Dataset

Figure 2.1 displays the original DataFrame created from a student dictionary. It contains the columns StudentID, Name, and Marks for five students. The pd.DataFrame function converts the dictionary into a structured format that can be analysed using Pandas.

2.2 shows only the Name and Marks columns selected from the original DataFrame. This demonstrates column selection and allows the analysis to focus on the student names and their corresponding marks without displaying StudentID.

Displaying only name and marks columns

```

   Name  Marks
0  Hasnain    45
1   John    86
2   Alice    90
3   Sahil    67
4   Cook    97
```

Figure 2.2 Selected Name and Marks Columns

Displaying students with marks greater than 70

```

  StudentID   Name  Marks
1         11   John    86
2         12   Alice    90
4         14   Cook    97
```

Figure 2.3 Filtering student with marks > 70

Figure 2.3 presents the filtered records for students whose marks are greater than 70. The Boolean condition dataFrame["Marks"] > 70 retains John, Alice, and Cook because their marks satisfy the specified condition.

2.4 displays the updated DataFrame after adding the Grade column. Grades are assigned using conditional logic: A for marks of 90 or above, B for 80-89, C for 70-79, D for 60-69, and marks below 60.

Displaying the updated dataset

```

  StudentID   Name  Marks Grade
0         10  Hasnain    45     E
1         11   John    86     B
2         12   Alice    90     A
3         13   Sahil    67     D
4         14   Cook    97     A
```

Figure 2.4 Dataset after Adding the Grade Column

Displaying the sorted dataset

```

  StudentID   Name  Marks Grade
0         10  Hasnain    45     E
3         13   Sahil    67     D
1         11   John    86     B
2         12   Alice    90     A
4         14   Cook    97     A
```

2.5 Dataset Sorted by Marks in Ascending Order

Figure 2.5 shows the records sorted from the lowest to the highest marks using sort_values(). Hasnain appears first with 45 marks, while Cook appears last with 97 marks. The original row indexes are retained by Pandas.

**Output
Snapshots
With
Explanation/
Observations**

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

	Figure 2.6 displays the average marks calculated by using mean() function. The total marks of all students divided by five gives an average 77.0. <i>Figure 2.6 Average Marks</i>
Conclusions	This exercise successfully demonstrated basic student-data manipulation and analysis using Pandas. A DataFrame containing StudentID, Name, and Marks was created from a Python dictionary, and multiple operations were performed on the dataset. The results showed that: • The Name and Marks columns were selected successfully for focused analysis. • Students with marks greater than 70 were identified using Boolean filtering. • A Grade column was generated using conditional logic based on each student's marks. • The records were arranged in ascending order of marks using sort_values(). • The average marks of the five students were calculated as 77.0 using mean(). Overall, the exercise provided practical experience in DataFrame creation, column selection, row filtering, conditional transformation, sorting, and statistical aggregation. These operations are fundamental for data preprocessing and exploratory data analysis in Data Science and Machine Learning applications.
References	1. Wes McKinney, Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter, 3rd Edition, O'Reilly Media, 2022. 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab Link) https://colab.research.google.com/drive/1qBzvtH6ZEis8Jv20WDiYXKI4PXGSXcWJ?usp=sharing 4. Pandas DataFrame API Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html 5. Class lecture link: https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

Title	Data Manipulation and Analysis using the Starwars Dataset in R					
Problem Statement	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. 5. Normalize rating values.					
Objectives	• To understand movie data preprocessing using the Pandas library. • To create a dataset containing missing values. • To identify and handle missing values in a dataset. • To fill missing Rating values using the average rating. • To remove records with missing MovieID or Genre. • To convert the Rating column into integer data type. • To normalize movie rating values using Min-Max normalization. • To identify top-rated movies based on ratings.					
Dataset URL Link / File	MovieID	MovieTitle	Rating	Genre	ViewCount	Likes
	MV101	The Last Horizon	8.5	Science Fiction	125000	11200
	MV102	City Lights		Comedy	98000	7500
	MV103	Hidden Truth	7.8	Thriller	110000	8900
	None	Lost Kingdom	9.1	Fantasy	150000	13500
	MV105	Digital Dreams	8.9	Science Fiction	175000	16000
	MV106	Silent River		Drama	87000	6100
	MV107	Future Code	9.3	Science Fiction	210000	19800
	MV108	Broken Compass	6.7		65000	4200
	MV109	Midnight Journey	8.1	Adventure	132000	10400
	MV110	Laugh Again	7.5	Comedy	105000	8100
	MV111	Ocean Mystery	8.8	Mystery	143000	12100
	MV112	Final Mission		Action	190000	17400

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	MV113	Parallel Lives	8.4	Drama	118000	9600
	MV114	Wild Trails	7.2	Adventure	99000	7200
	MV115	The Great Escape	9.0	Action	205000	18800
Functions Details	Sl. No.	Function	Package	Description of function		
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the movie dataset stored in dictionary format.		
	2	isnull()	Pandas	Identifies missing values present in the movie dataset.		
	3	mean()	Pandas	Fills missing Rating values using the calculated average rating.		
	4	dropna()	Pandas	Removes rows containing missing MovieID or Genre values.		
	5	pd.to_numeric()	Pandas	Converts Rating values into numeric format and handles invalid entries.		
	6	round()	Pandas	Rounds decimal Rating values before converting them into integers.		
	7	astype()	Pandas	Converts the Rating column into integer data type.		
	8	min()	Pandas	Finds the minimum Rating value required for normalization.		
	9	max()	Pandas	Finds the maximum Rating value required for normalization.		
	10	groupby()	Pandas	Groups movie records according to Genre for genre-wise analysis.		
	11	sort_values()	Pandas	Sorts movies based on rating, popularity score or number of views.		
	12	fillna()	Pandas	Fills missing Rating values using the calculated average rating.		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import the Pandas library and create the movie dataset in dictionary format.					
	Step 2: Convert the dictionary into a Pandas DataFrame using pd.DataFrame().					
	Step 3: Display the original dataset containing missing values.					
	Step 4: Calculate the average value of the Rating column using the mean() function.					
	Step 5: Fill the missing Rating values using fillna() with the calculated average rating.					

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	Step 6: Remove rows containing missing MovieID or Genre using the dropna() function. Step 7: Convert the Rating column into numeric format using pd.to_numeric() and then convert it into integer type using round() and astype(int). Step 8: Normalize the Rating values using the Min-Max Normalization formula: Normalized Rating = (Rating – Minimum Rating) / (Maximum Rating – Minimum Rating) Step 9: Calculate the Engagement Rate and Popularity Score for each movie based on views, likes and ratings. Step 10: Sort the movies according to Popularity Score and Rating using sort_values() and assign popularity ranks. Step 11: Generate a genre-wise report using the groupby() function to compare movie performance across different genres. Step 12: Recommend the top movies based on selected genre and minimum rating, generate the automated analytics report, and save the processed results for future analysis.
Program	<pre>import pandas as pd # ----- # 1. CREATE MOVIE DATASET IN DICTIONARY FORMAT # ----- movie_data = { "MovieID": ["MV101", "MV102", "MV103", None, "MV105", "MV106", "MV107", "MV108", "MV109", "MV110", "MV111", "MV112", "MV113", "MV114", "MV115"], "MovieTitle": ["The Last Horizon", "City Lights", "Hidden Truth", "Lost Kingdom", "Digital Dreams", "Silent River", "Future Code", "Broken Compass", "Midnight Journey",] }</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Laugh Again",
"Ocean Mystery",
"Final Mission",
"Parallel Lives",
"Wild Trails",
"The Great Escape"
],

"Rating": [
    8.5, None, 7.8, 9.1, 8.9,
    None, 9.3, 6.7, 8.1, 7.5,
    8.8, None, 8.4, 7.2, 9.0
],

"Genre": [
    "Science Fiction",
    "Comedy",
    "Thriller",
    "Fantasy",
    "Science Fiction",
    "Drama",
    "Science Fiction",
    None,
    "Adventure",
    "Comedy",
    "Mystery",
    "Action",
    "Drama",
    "Adventure",
    "Action"
],

"ViewCount": [
    125000, 98000, 110000, 150000, 175000,
    87000, 210000, 65000, 132000, 105000,
    143000, 190000, 118000, 99000, 205000
],

"Likes": [
    11200, 7500, 8900, 13500, 16000,
    6100, 19800, 4200, 10400, 8100,
    12100, 17400, 9600, 7200, 18800
]
}

# -----
# 2. CONVERT DICTIONARY INTO DATAFRAME
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# -----  
movies = pd.DataFrame(movie_data)  
  
print("\nORIGINAL MOVIE DATASET")  
print("=" * 90)  
print(movies.to_string(index=False))  
  
# -----  
# 3. CALCULATE AVERAGE RATING  
# -----  
average_rating = movies["Rating"].mean()  
  
print("\nAVERAGE RATING")  
print("=" * 90)  
print("Average Rating:", round(average_rating, 2))  
  
# -----  
# 4. FILL MISSING RATING VALUES  
# -----  
movies["Rating"] = movies["Rating"].fillna(average_rating)  
  
print("\nDATASET AFTER FILLING MISSING RATING VALUES")  
print("=" * 90)  
print(  
    movies[  
        ["MovieID", "MovieTitle", "Rating", "Genre"]  
    ].to_string(index=False)  
)  
  
# -----  
# 5. REMOVE ROWS WITH MISSING MOVIEID OR GENRE  
# -----  
movies = movies.dropna(subset=["MovieID", "Genre"])  
  
print("\nDATASET AFTER REMOVING MISSING MOVIEID OR  
GENRE")  
print("=" * 90)  
print(  
    movies[  
        ["MovieID", "MovieTitle", "Rating", "Genre"]  
    ].to_string(index=False)  
)  
  
# -----
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# 6. CONVERT RATING INTO INTEGER TYPE
# -----
movies["Rating"] = movies["Rating"].round().astype(int)

print("\nRATING CONVERTED INTO INTEGER TYPE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating"]
    ].to_string(index=False)
)

print("\nRating Data Type:", movies["Rating"].dtype)

# -----
# 7. NORMALIZE RATING VALUES
# Min-Max Normalization
# -----
minimum_rating = movies["Rating"].min()
maximum_rating = movies["Rating"].max()

movies["NormalizedRating"] =
    ( (movies["Rating"] - minimum_rating)
      / (maximum_rating - minimum_rating)
    ).round(3)

print("\nNORMALIZED RATING VALUES")
print("=" * 90)
print(
    movies[
        [
            "MovieID",
            "MovieTitle",
            "Rating",
            "NormalizedRating"
        ]
    ].to_string(index=False)
)

# -----
# 8. CALCULATE ENGAGEMENT RATE
# -----
movies["EngagementRate"] = (
    movies["Likes"] / movies["ViewCount"] * 100
).round(2)
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nVIEWER ENGAGEMENT")
print("=" * 90)
print(
    movies[
        [
            "MovieTitle",
            "ViewCount",
            "Likes",
            "EngagementRate"
        ]
    ].to_string(index=False)
)

# -----
# 9. CALCULATE POPULARITY SCORE
# -----
maximum_engagement = movies["EngagementRate"].max()

movies["NormalizedEngagement"] = (
    movies["EngagementRate"] / maximum_engagement
).round(3)

movies["PopularityScore"] =
    ( movies["NormalizedRating"] * 0.60
      + movies["NormalizedEngagement"] * 0.40
    ).round(3)

# -----
# 10. SORT AND RANK MOVIES
# -----
movies =
    movies.sort_values( by=["Popular
        ityScore", "Rating"],
        ascending=[False, False]
    ).reset_index(drop=True)

movies["PopularityRank"] = range(1, len(movies) + 1)

print("\nTOP-RATED AND POPULAR MOVIES")
print("=" * 90)
print(
    movies[
        [
            "PopularityRank",
            "MovieID",
            "MovieTitle",
            "Genre",
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Rating",
"ViewCount",
"PopularityScore"
]
].to_string(index=False)
)

# -----
# 11. GENERATE GENRE-WISE REPORT
# -----
genre_report = movies.groupby(
    "Genre",
    as_index=False
).agg(
    MovieCount=("MovieID", "count"),
    AverageRating=("Rating", "mean"),
    HighestRating=("Rating", "max"),
    TotalViews=("ViewCount", "sum"),
    TotalLikes=("Likes", "sum")
)

genre_report["AverageRating"] = (
    genre_report["AverageRating"].round(2)
)

genre_report =
    genre_report.sort_values( by="AverageRating", ascending=False
)

print("\nGENRE-WISE MOVIE REPORT")
print("=" * 90)
print(genre_report.to_string(index=False))

# -----
# 12. SIMPLE MOVIE RECOMMENDATION
# -----
preferred_genre = "Science Fiction"
minimum_required_rating = 7

recommended_movies =
    movies[ (movies["Genre"] ==
    preferred_genre)
    & (movies["Rating"] >= minimum_required_rating)
].sort_values( by="Popularity
Score", ascending=False
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
)

print("\nMOVIE RECOMMENDATIONS")
print("=" * 90)
print("Preferred Genre:", preferred_genre)
print("Minimum Rating:", minimum_required_rating)

print(
    recommended_movies[
        [
            "MovieID",
            "MovieTitle",
            "Genre",
            "Rating",
            "PopularityScore"
        ]
    ].to_string(index=False)
)

# -----
# 13. AUTOMATED ANALYTICS REPORT
# -----
top_movie = movies.iloc[0]

most_viewed_movie = movies.loc[
    movies["ViewCount"].idxmax()
]

best_genre = genre_report.iloc[0]

print("\nAUTOMATED MOVIE ANALYTICS REPORT")
print("=" * 90)

print("Total Movies Analysed:", len(movies))
print("Average Movie Rating:", round(movies["Rating"].mean(), 2))
print("Total Genres:", movies["Genre"].nunique())

print("\nTop-Rated and Most Popular Movie")
print("Movie Title:", top_movie["MovieTitle"])
print("Genre:", top_movie["Genre"])
print("Rating:", top_movie["Rating"])
print("Popularity Score:", top_movie["PopularityScore"])

print("\nMost Viewed Movie")
print("Movie Title:", most_viewed_movie["MovieTitle"])
print("Views:", most_viewed_movie["ViewCount"])
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nBest Performing Genre")
print("Genre:", best_genre["Genre"])
print("Average Rating:", best_genre["AverageRating"])

# -----
# 14. SAVE RESULTS
# -----

movies.to_csv( "processed_movie_data.csv", index=False
)

genre_report.to_csv( "genre_wise_report.csv",
                    index=False
)

print("\nFILES CREATED")
print("=" * 90)
print("processed_movie_data.csv")
print("genre_wise_report.csv")
```

**Output
Snapshots
With
Explanation/
Observations**

```
1. ORIGINAL DATASET WITH MISSING VALUES
=====
MovieID  MovieTitle  Rating  Genre  ViewCount  Likes
MV101 The Last Horizon  8.5  Science Fiction  125000  11200
MV102 City Lights  NaN  Comedy  98000  7500
MV103 Hidden Truth  7.8  Thriller  110000  8900
      NaN  Lost Kingdom  9.1  Fantasy  150000  13500
MV105 Digital Dreams  8.9  Science Fiction  175000  16000
MV106 Silent River  NaN  Drama  87000  6100
MV107 Future Code  9.3  Science Fiction  210000  19000
MV108 Broken Compass  6.7  NaN  65000  4200
MV109 Midnight Journey  8.1  Adventure  132000  10400
MV110 Laugh Again  7.5  Comedy  105000  8100
MV111 Ocean Mystery  8.8  Mystery  143000  12100
MV112 Final Mission  NaN  Action  190000  17400
MV113 Parallel Lives  8.4  Drama  118000  9600
MV114 Wild Trails  7.2  Adventure  99000  7200
MV115 The Great Escape  9.0  Action  205000  18800
```

Figure 1.1 Original dataset printed after creating the DataFrame (containing missing values).

The average rating was calculated successfully and was used to replace all missing Rating values.

```
2. FILL MISSING RATING USING AVERAGE RATING
=====
Average Rating used: 8.28
MovieID  MovieTitle  Rating
MV107 Future Code  9
MV115 The Great Escape  9
MV105 Digital Dreams  9
MV111 Ocean Mystery  9
MV112 Final Mission  8
MV101 The Last Horizon  8
MV113 Parallel Lives  8
MV103 Hidden Truth  8
MV109 Midnight Journey  8
MV110 Laugh Again  8
MV102 City Lights  8
MV106 Silent River  8
MV114 Wild Trails  7
```

Figure 1.2 Output showing the calculated average rating.

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
=====
3. REMOVE ROWS WITH MISSING MOVIEID OR GENRE
=====
MovieID  MovieTitle  Genre  Rating
MV107    Future Code  Science Fiction  9
MV115    The Great Escape  Action  9
MV105    Digital Dreams  Science Fiction  9
MV111    Ocean Mystery  Mystery  9
MV112    Final Mission  Action  8
MV101    The Last Horizon  Science Fiction  8
MV113    Parallel Lives  Drama  8
MV103    Hidden Truth  Thriller  8
MV109    Midnight Journey  Adventure  8
MV110    Laugh Again  Comedy  8
MV102    City Lights  Comedy  8
MV106    Silent River  Drama  8
MV114    Wild Trails  Adventure  7
=====
```

Figure 1.3 Dataset after fillna() operation.

Rating values were converted into integer type and normalized using the Min-Max Normalization technique.

```
=====
5. NORMALIZE RATING VALUES
=====
MovieID  MovieTitle  Rating  NormalizedRating
MV107    Future Code  9        1.0
MV115    The Great Escape  9        1.0
MV105    Digital Dreams  9        1.0
MV111    Ocean Mystery  9        1.0
MV112    Final Mission  8        0.5
MV101    The Last Horizon  8        0.5
MV113    Parallel Lives  8        0.5
MV103    Hidden Truth  8        0.5
MV109    Midnight Journey  8        0.5
MV110    Laugh Again  8        0.5
MV102    City Lights  8        0.5
MV106    Silent River  8        0.5
MV114    Wild Trails  7        0.0
=====
```

Figure 1.4 Integer Rating output along with the **NormalizedRating** column.

```
=====
TOP-RATED AND POPULAR MOVIES
=====
PopularityRank  MovieID  MovieTitle  Genre  Rating  ViewCount  PopularityScore
1  MV107    Future Code  Science Fiction  9  210000  1.000
2  MV115    The Great Escape  Action  9  205000  0.989
3  MV105    Digital Dreams  Science Fiction  9  175000  0.988
4  MV111    Ocean Mystery  Mystery  9  143000  0.959
5  MV112    Final Mission  Action  8  190000  0.689
6  MV101    The Last Horizon  Science Fiction  8  125000  0.680
7  MV113    Parallel Lives  Drama  8  118000  0.645
8  MV103    Hidden Truth  Thriller  8  110000  0.643
9  MV109    Midnight Journey  Adventure  8  132000  0.634
10 MV110    Laugh Again  Comedy  8  105000  0.627
11 MV102    City Lights  Comedy  8  98000  0.624
12 MV106    Silent River  Drama  8  87000  0.597
13 MV114    Wild Trails  Adventure  7  99000  0.308
=====
```

Figure 1.5 Sorted movie list showing **Popularity Rank**.

Movies were ranked successfully based on their popularity score, allowing easy identification of top-performing movies.

Conclusions

The Movie Rating Data Preprocessing project was successfully implemented using the Pandas library. The dataset was created with missing values, and the missing Rating values were replaced using the average rating. Records with missing MovieID or Genre were removed to improve data quality. The Rating column was converted into integer type, and the values were normalized using the Min-Max normalization technique. The project also identified top-rated movies, generated genre-wise reports, analysed viewer preferences, recommended

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	movies based on genre and rating, and visualized movie popularity trends. This exercise provided practical knowledge of data preprocessing, data analysis, and dashboard development using Python, Pandas, and Streamlit.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%202/EXERCISE_PROGRAM.py) 3. Class lecture link https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view